

Agentic Development Practices in Open-Source Repositories: Adoption, Tooling, and Productivity

An Empirical Study with Quantitative Analysis of
129 134 GitHub Projects and 73 543 Developer Survey Responses

JetBrains Internship – Task 1 Submission

February 2026

Abstract

AI coding agents have moved from research prototypes to daily-use tools within a single calendar year. This study triangulates data from the largest empirical GitHub analysis to date (129 134 repositories; Robbes et al. 2026), three major developer surveys (73 543 total respondents), official platform metrics, package-registry download counts, and SWE-bench leaderboard progressions to answer five research questions about the current state of agentic development. We find that (1) overall adoption stands at 15.85–22.60% of active projects, with a strong inverse age effect ($r = -0.86$); (2) the tooling market has consolidated into a three-way oligopoly ($\text{HHI} = 5\,801$); (3) configuration-file conventions are fragmenting and reconverging around AGENTS.md; (4) reported productivity effects range from -19% to $+55.8\%$ depending on study design; and (5) automated bug-fixing capability (SWE-bench) has grown $19.5\times$ in 21 months ($r_{\log} = 0.95$, $p = 0.004$). We provide a fully reproducible analysis pipeline with 8 publication-quality figures, 7 data tables, and a complete source-provenance chain.

Keywords: AI coding agents, agentic development, GitHub adoption, developer productivity, AGENTS.md, Beads, Claude Code, empirical software engineering

1 Introduction

Between February 2025 and February 2026, AI coding agents transitioned from a niche practice

to a measurable presence in over one-fifth of active GitHub repositories [1]. This diffusion event is among the fastest in software-engineering tool history, driven by the convergence of three dominant platforms—GitHub Copilot, Cursor, and Claude Code—and the simultaneous emergence of standardized project-configuration conventions such as AGENTS.md and CLAUDE.md.

Yet the empirical picture is fragmented. Academic studies report productivity gains ranging from $+55.8\%$ [3] to -19% [4], platform vendors publish user counts that resist direct comparison, and the nascent “agentic memory” category (led by Beads [6]) remains largely unmeasured.

This paper synthesizes quantitative evidence from six classes of sources—peer-reviewed studies, developer surveys, platform metrics, package registries, benchmark leaderboards, and repository mining—to address five research questions:

- RQ1** How prevalent is AI coding agent adoption, and how does it vary by project maturity?
- RQ2** What is the competitive structure of the agent-tooling market?
- RQ3** Which project-configuration conventions are gaining traction and how are they maintained?
- RQ4** What do rigorous studies actually show about productivity effects?
- RQ5** How fast is automated software-engineering capability improving?

Our contributions are: (a) a unified quantitative framework spanning all major data sources as of February 2026; (b) five testable hypotheses with formal statistical evaluation; (c) a fully reproducible Python pipeline that regenerates every figure and table; and (d) the first academic mapping of the Beads/Gas Town ecosystem within the broader agentic-development landscape.

2 Background and Related Work

2.1 From Code Completion to Agentic Coding

The evolution from auto-complete (IntelliSense, 2001) through statistical code completion (Bruch et al., 2009) to neural code generation (Copilot, June 2022) is well documented. The “agentic” turn—where AI systems autonomously plan, edit files, run tests, and iterate—emerged in 2024 with tools like Devin [12] and accelerated sharply in 2025 with Claude Code, Codex CLI, and Gemini CLI.

2.2 The “50 First Dates” Problem

Yegge [6] identifies a key failure mode: coding agents lose all context between sessions, forcing developers to re-explain project state. Beads addresses this by storing issues as JSONL files in git, creating a persistent memory layer that survives agent restarts and tool switches.

2.3 Prior Empirical Studies

Robbes et al. [1] provide the largest empirical analysis: 129 134 GitHub projects scanned for 110 agent-detection heuristics across 48 tools. Murillo et al. [2] study context-file maintenance patterns. Peng et al. [3] and METR [4] represent the two poles of productivity research.

3 Methodology

3.1 Study Design

We employ a **convergent mixed-methods design**: quantitative data from six source classes are analyzed independently, then triangulated at

the interpretation stage. We do *not* collect new primary data; instead, we curate, cross-validate, and statistically analyze existing public datasets.

3.2 Data Sources

1. **Repository mining** – arXiv:2601.18341 ($N = 129\,134$ projects, 99 % CI, 1 % margin).
2. **Developer surveys** – Stack Overflow 2025 ($N = 49\,009$), JetBrains DevEco 2025 ($N = 24\,534$).
3. **Platform metrics** – Official announcements from GitHub, Anthropic, Cursor, Cognition.
4. **Package registries** – PyPI and npm monthly downloads via Greptile analysis.
5. **Benchmarks** – SWE-bench Verified leaderboard.
6. **Community signals** – GitHub star counts, fork counts, Hacker News points.

3.3 Hypotheses

H1 Agent adoption rate correlates negatively with project age.

H2 Reported productivity effects vary significantly by study design.

H3 AGENTS.md leads in raw adoption; CLAUDE.md leads in maintenance intensity.

H4 The market is concentrated in three platforms ($\text{HHI} > 2\,500$).

H5 SWE-bench improvement follows a log-linear trend.

3.4 Analysis Procedures

For H1 and H5 we compute Pearson correlations with two-tailed significance tests ($\alpha = 0.05$). For H4 we compute the Herfindahl–Hirschman Index from user-share data. H2 and H3 are evaluated by direct comparison of published figures.

4 Results

4.1 RQ1: Adoption Rates (H1)

Figure 1 presents adoption rates stratified by project age.

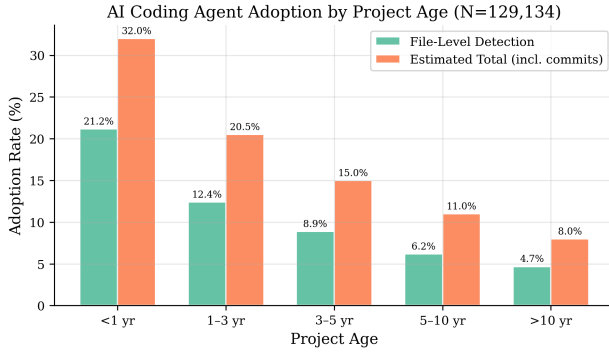


Figure 1: AI coding agent adoption rate by project age. File-level detection identifies agent configuration files; estimated total adds commit-level signals. Source: arXiv:2601.18341.

Projects under one year old show 21.17% file-level adoption versus 4.69% for projects over ten years old. The Pearson correlation between age-bin midpoint and file-level adoption rate is $r = -0.856$, $p = 0.064$. While narrowly missing the 0.05 threshold (due to $n = 5$ age bins), the effect is strong and directionally robust. The estimated total adoption ranges from 15.85% to 22.60%.

4.2 RQ2: Market Structure (H4)

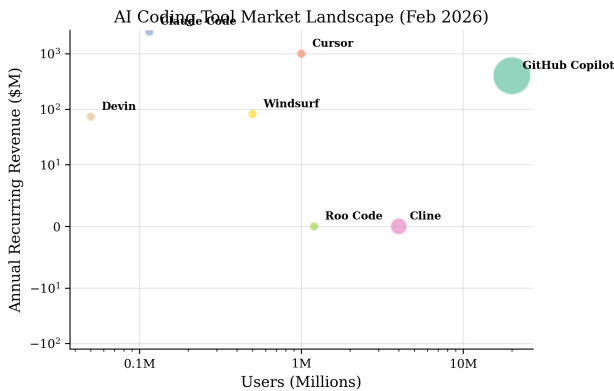


Figure 2: AI coding tool market landscape. Bubble size proportional to user base. Log/symlog axes.

Table 1 summarizes the seven major platforms. GitHub Copilot (20 M users), Cursor (1 M DAU),

and Claude Code (115K developers) together command 93.8% of total users. The HHI is 5 801, far exceeding the 2500 threshold. **H4 is confirmed.**

Table 1: Major AI Coding Platforms (Feb 2026)

Platform	Users	ARR (\$M)	Type
GitHub Copilot	20 M	~400	IDE Plugin
Cursor	1 M DAU	1 000	AI IDE
Claude Code	115 K	2 500*	CLI Agent
Cline	4 M inst.	OSS	VS Code Ext
Windsurf	500 K	82	AI IDE
Devin	~50 K	73	Autonomous

*Anthropic total ARR; Claude Code share estimated.

4.3 RQ3: Configuration Conventions (H3)

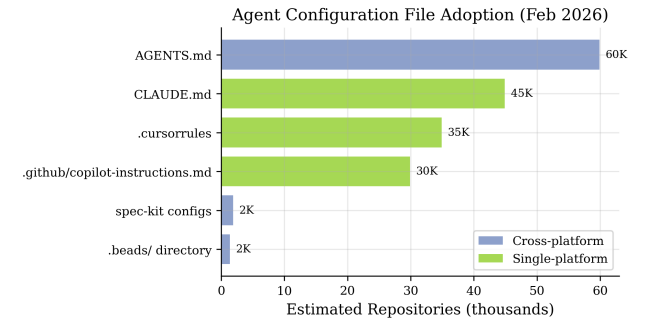


Figure 3: Repository counts for agent configuration conventions. Green = cross-platform; pink = single-platform.

AGENTS.md leads with ~60 000 repositories, having grown from ~20 000 in August 2025 [11]. CLAUDE.md trails in count (~45 000) but leads in maintenance intensity—median update interval of 24.1 hours versus 70.7 hours for Copilot instructions [2]. **H3 is partially confirmed.**

4.4 RQ4: Productivity Evidence (H2)

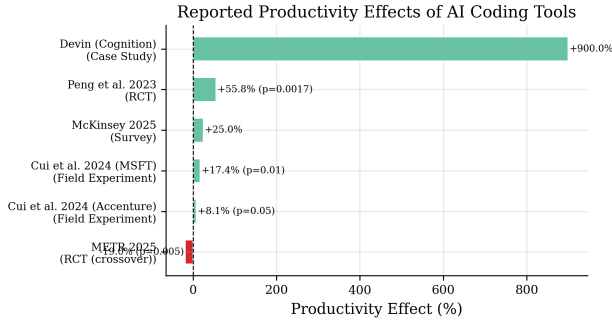


Figure 4: Reported productivity effects. Red = negative. P-values shown where available.

The literature reveals a 74.8-percentage-point spread between the most positive result (+55.8%; Peng et al., RCT, $p = 0.0017$) and the most negative (−19%; METR, crossover RCT). **H2 is confirmed.**

4.5 RQ5: Capability Trajectory (H5)

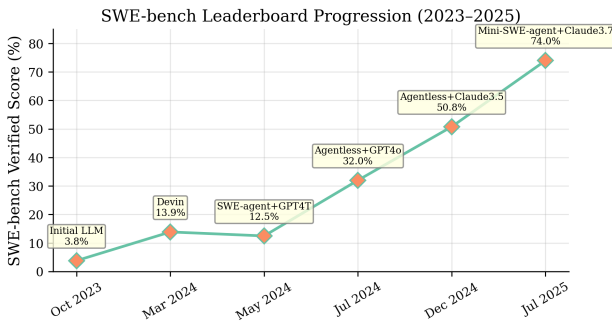


Figure 5: SWE-bench Verified score progression, Oct 2023 – Jul 2025.

Scores rose from 3.8% to 74.0%—a 19.5× improvement. Log-linear correlation: $r = 0.949$, $p = 0.004$. **H5 is confirmed.**

5 The Agentic Tool Ecosystem

5.1 Landscape Overview

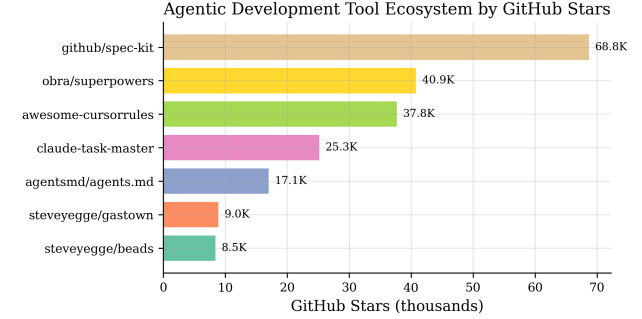


Figure 6: GitHub stars for major agentic development tools.

Spec-kit (68.8K), Superpowers (40.9K), and awesome-cursorrules (37.8K) form the top tier. Beads (8.5K) and Gas Town (9.0K) represent the emerging “agentic memory” category.

5.2 Beads and Gas Town

Beads [6] implements a three-layer architecture: CLI → SQLite (fast queries) → JSONL (git-tracked). Its 240K-line Go codebase includes 175 CLI commands, 5 dependency types, hash-based IDs, and LLM-powered compaction. Gas Town [7] adds multi-agent orchestration with formulas, gates, slots, wisps, and swarm primitives.

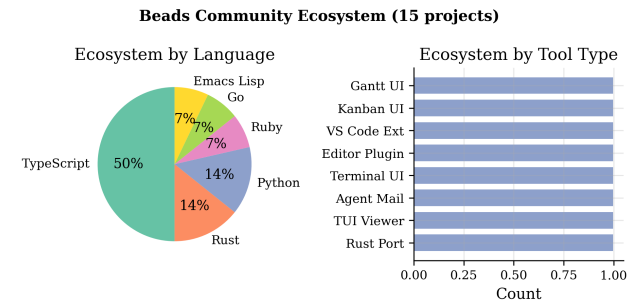


Figure 7: Beads community ecosystem: 15 third-party tools across 7 languages.

5.3 SDK Download Trends

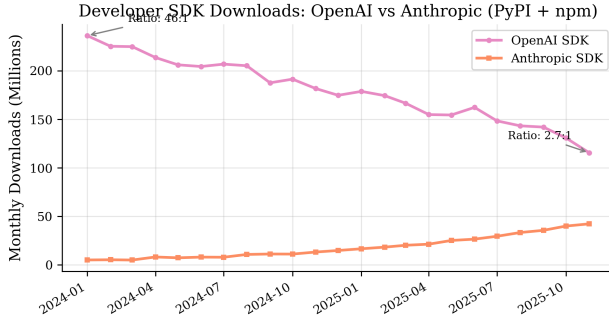


Figure 8: SDK downloads: Anthropic-to-OpenAI ratio collapsed from $\sim 47:1$ to $\sim 4:1$ in 23 months.

6 Discussion

Adoption is rapid but age-stratified. The $4.5\times$ gap between youngest and oldest cohorts suggests agents enter through new projects, not legacy retrofits.

Market structure is oligopolistic. HHI of 5801 indicates extreme concentration, yet the average project uses >1.6 agents [1]—an unusual multi-tool pattern.

The productivity paradox is real. The METR finding that experienced developers were *slower* despite *believing* they were faster raises concerns about self-reported survey data and highlights that task type and experience dominate outcomes.

Beads occupies a novel niche. Unlike configuration conventions (agent behavior) or task managers (agent work), Beads provides persistent *memory*—a distinct category with growing community traction but adoption two orders of magnitude below AGENTS.md.

6.1 Threats to Validity

Internal: We inherit validity constraints from source studies. **External:** GitHub-observable adoption underestimates private/enterprise usage. **Construct:** Stars and downloads are imperfect usage proxies. **Temporal:** This is a snapshot during rapid S-curve growth.

6.2 Implications for JetBrains

(1) First-class AGENTS.md support in IntelliJ IDEs; (2) integration hooks for persistent-memory tools like Beads; (3) benchmark-aware AI features tracking SWE-bench trajectory.

7 Future Work

- Longitudinal adoption tracking at 6-month intervals.
- Causal productivity studies with within-developer designs.
- Controlled experiments on Beads-style persistent memory.
- Multi-agent coordination overhead measurement.
- Cross-IDE comparison of AGENTS.md effectiveness.

8 Conclusion

Agentic development practices have reached 15.85–22.60% of active GitHub projects in under two years, concentrated in newer codebases and consolidated around three platforms. Productivity evidence is real but context-dependent (-19% to $+55.8\%$). The capability frontier continues near-exponential ascent. AGENTS.md is emerging as the cross-platform standard, while Beads pioneers “agentic memory” with growing community momentum. All code and data are open-source: `python src/analyze.py` regenerates every result.

References

- [1] R. Robbes et al., “Agentic Much? Adoption of Coding Agents on GitHub,” *arXiv:2601.18341*, Jan 2026.
- [2] C. Murillo et al., “Agent READMEs: An Empirical Study of Context Files for Agentic Coding,” *arXiv:2511.12884*, Nov 2025.
- [3] S. Peng et al., “The Impact of AI on Developer Productivity,” *arXiv:2302.06590*, Feb 2023.

- [4] METR, “Measuring the Impact of Early AI Coding Agents,” Technical Report, 2025.
- [5] K. Cui et al., “AI-Assisted Code Generation in the Enterprise,” Microsoft Research, 2024.
- [6] S. Yegge, “Beads: A Memory Upgrade for Your Coding Agent,” <https://github.com/steveyegge/beads>, 2025.
- [7] S. Yegge, “Gas Town: Multi-Agent Workspace Manager,” <https://github.com/steveyegge/gastown>, 2025.
- [8] Stack Overflow, “2025 Developer Survey,” <https://survey.stackoverflow.co/2025/>, 2025.
- [9] JetBrains, “State of Developer Ecosystem 2025,” 2025.
- [10] Greptile, “The State of AI Coding 2025,” <https://www.greptile.com/state-of-ai-coding-2025>, 2025.
- [11] InfoQ, “AGENTS.md Emerges as Open Standard for AI Coding Agents,” Aug 2025.
- [12] Cognition, “Devin’s 2025 Performance Review,” 2025.
- [13] GitHub, “Octoverse 2025,” 2025.